

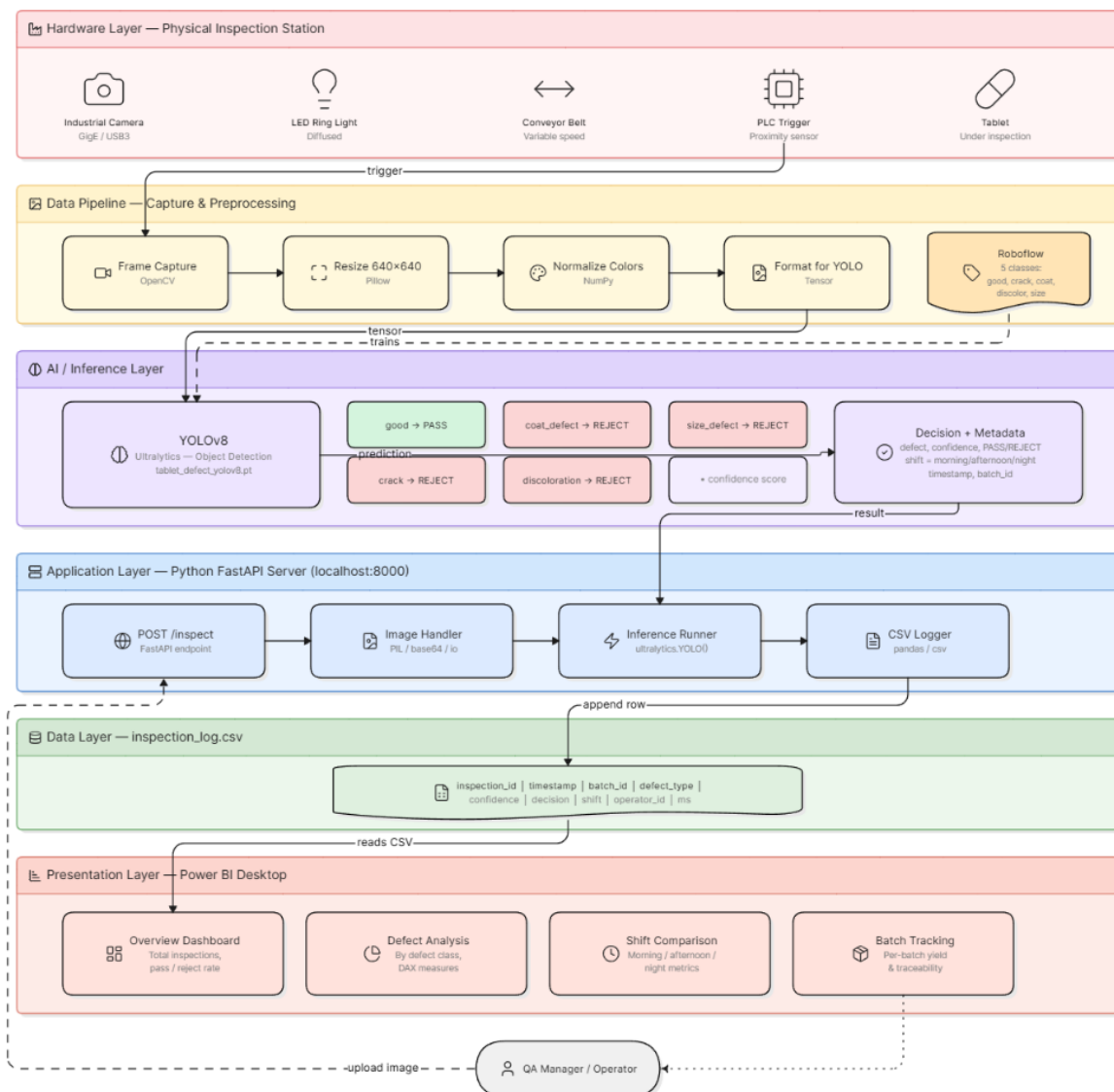
AI and Edge Computing L&T Internship

Batch-5

- Chalapala Sowmya[CB.EN.U4ELC23014]
- Lalitha Saranya Pathi[CB.EN.U4ELC23025]
- Nakkala Deepak Reddy [CB.EN.U4ELC23028]
- Naga Sriyasantoshi Puranam[CB.EN.U4ELC23060]

Day-1[05-06-2026]

System Architecture



Workflow

Step-1:Collect tablet images

We collect real photos of tablets good ones and defective ones from the production line or a test setup.

Step-2: Label images in Roboflow

Labelling means drawing a box around each defect in every photo and naming it. This teaches the AI what each defect looks like. We do it using Roboflow.

Step-3:Train the YOLOv8 model

Training means feeding all our labelled photos to YOLOv8 and letting it learn. After training, the model can look at a brand new tablet photo and identify defects it has never seen before.

Step-4: Build the Python API (FastAPI)

The API is the bridge between Power BI and our YOLOv8 model. Think of it as a waiter Power BI gives it an image, it takes that image to YOLOv8, gets the result, and brings the answer back.

Step-5: Test the API with Postman

Before connecting Power BI, we should always test our API manually. Postman is a free tool that lets us send a fake request to our API just like Power BI would, so we can make sure everything works first

Step-6: Set up the folder watcher

Power BI cannot save files to our PC directly. So instead: the user drops a tablet image into a Windows folder, and a Python script (watcher.py) automatically detects it and sends it to the API. Simple and reliable.

Step-7: Understanding the full image journey

- 1.User copies or moves tablet photo into C:/tablet_images/
- 2.watcher.py detects new file → reads it as bytes
- 3.watcher.py sends POST request to <http://localhost:8000/inspect> with image + shift + batch_id
- 4.FastAPI receives the request → loads image into memory

5. YOLOv8 model runs inference → returns bounding boxes + class + confidence
6. FastAPI formats result as JSON → appends row to results_log.csv
7. FastAPI sends JSON result back to watcher.py → watcher prints it in terminal
8. Power BI reads results_log.csv on next refresh → dashboard updates

Step-8: Connect Power BI to the CSV

Power BI reads our results_log.csv file and turns rows of data into visual charts. Every time the CSV gets a new row (a new inspection), Power BI picks it up on the next refresh.

Step-9: Build the 4 dashboard pages

Now we build the visual screens inside Power BI. Each page shows a different view of the inspection data. Users can click between pages to see different insights.

Step-10: Run the complete system

1. Open terminal 1 → cd to project folder → run: `uvicorn api:app --reload --port 8000`
2. Open terminal 2 → run: `python watcher.py` (make sure SHIFT is set correctly)
3. Open Power BI Desktop → open tablet_dashboard.pbix
4. Drop a tablet image into C:/tablet_images/
5. Watch terminal 2 — result prints within 2 seconds
6. Click Refresh in Power BI — the dashboard updates with the new result
7. At shift change: stop watcher.py → change SHIFT variable → restart watcher.py
8. At end of day: export Page 4 as PDF → submit as GMP compliance report

Conveyor Speed:

The end-to-end inspection cycle for a single tablet breaks down as follows: image capture takes approximately 3 seconds, transmission to the API takes 0.2 seconds, YOLOv8 inference takes 0.05 seconds, and CSV logging with Power BI refresh takes 1–2 seconds. This gives a total cycle time of 4.25 to 5.25 seconds per tablet.

Given a tablet spacing of 10 cm on the conveyor belt, the belt speed works out to approximately 0.019–0.024 m/s to ensure each tablet is captured within its inspection window. At this cycle time, the system can process around 14 tablets per minute.

Since the belt is running at low speed and the camera operates at a fixed trigger per tablet, a 30–60 fps camera is sufficient. The short exposure time at this frame rate ensures there is no motion blur on the tablet surface, keeping the image sharp enough for accurate defect detection at the 640×640 input resolution used by YOLOv8.

Key figures at a glance:

Parameter	Value
Total cycle time	4.25 – 5.25 seconds
Belt speed	0.019 – 0.024 m/s
Tablet spacing	10 cm
Throughput	~14 tablets / min
Camera frame rate	30 – 60 fps
Image resolution	640 × 640 px

Challenges & Mitigations

1. **Low Lighting** Inconsistent or poor lighting on the production line can reduce image quality and cause the model to misclassify tablets. To address this, data augmentation techniques are applied during training including contrast adjustment, brightness enhancement, and shadow simulation so the model learns to handle varying lighting conditions robustly.
2. **Unknown Objects Causing Camera Interruption** Foreign objects such as dust particles or insects occasionally pass in front of the camera, triggering an unintended capture. Since these objects bear no resemblance to any known defect class, the model returns a low confidence score that falls below the defined threshold of 85%. Any result that does not meet this threshold is automatically flagged and moved to an unknown folder for human review, preventing false classifications from entering the results log.
3. **Motion Blur and Camera Misalignment** Motion blur is mitigated by the camera and belt parameters defined above specifically the 30–60 fps frame rate and controlled belt speed of 0.019–0.024 m/s which together ensure each tablet is captured sharply within its inspection window. In the event of physical misalignment of the camera, the conveyor line is paused and human intervention is triggered to recalibrate the setup before inspection resumes.

4. Change of Batch Composition If the tablet formulation, shape, or coating changes after a period of production, the existing model may no longer perform accurately on the new variant. To handle this, the model undergoes retraining whenever a significant batch change occurs. Additionally, the system is designed to support active learning throughout its operational life continuously incorporating newly flagged or uncertain samples to maintain and improve accuracy over time without requiring a full retrain from scratch.